

Dynamical System Modeling of Super Smash Bros. Melee: Attractor Manifolds, Biomechanical Telemetry, and Stochastic Analysis

1. Introduction: The Paradigm Shift in Competitive Modeling

The analytical landscape of competitive gaming has historically been dominated by discrete computational models, primarily viewing player interactions through the lens of game theory, finite state machines, or Markovian state transitions. In traditional fighting games, where movement is often restricted to fixed-distance spatial increments, rigid grid-based arenas, or strictly programmed frame-data sequences, such discrete categorization is mathematically sufficient for high-level analysis. In these classical environments, an interaction can be modeled as a turn-based exchange of predefined actions, akin to a highly complex iteration of rock-paper-scissors. However, *Super Smash Bros. Melee* (SSBM), released for the Nintendo GameCube in 2001, presents a massive paradigm shift in competitive software design due to its physics-heavy engine, sub-frame collision detection, and continuous analog control mechanisms.¹

The movement in this specialized environment is not a series of discrete, teleporting jumps between predefined static states; rather, it is a fluid, high-velocity exchange governed by the conservation of simulated momentum, variable friction coefficients, and analog gravity acceleration. Consequently, high-level player behavior in this specific title more closely resembles the continuous dynamics of a physical sport, fluid mechanics, or a biological system than it does a classical digital video game. To capture the profound complexity of this interaction, it is mathematically necessary to move beyond simple frame-data parsing and adopt a comprehensive framework based on non-linear dynamics and chaos theory.

In competitive SSBM, the "neutral game" refers to the phase where neither player possesses a distinct positional or frame-based advantage over the other.¹ Players utilize rapid, highly technical micro-movements—such as dash-dancing, wavedashing, and wavelanding—to weave in and out of the opponent's threat ranges. Previous analyses have consistently struggled to quantify this highly volatile phase because they rely on discrete action parsing, which fails to capture the continuous nature of the spatial positioning. By applying chaos theory and non-linear dynamics, human intent and character physics can be modeled as a fluid system striving for structural stability.

By extracting sub-frame telemetry through modern parsing libraries, player coordinates and

instantaneous velocities can be mapped into a continuous phase space.¹ In this mathematical state space, the neutral game defies the traditional definition of a resting state.³ Instead, it is modeled as a metastable equilibrium characterized by strange attractors—specific, highly optimized spatial relationships maintained by players through continuous kinetic input. Furthermore, the transition from neutral to an advantage state (e.g., an offensive combo or edge-guard) is not merely a change in frame advantage, but a topological bifurcation in the system's governing dynamics, which can be quantified via Lyapunov exponents. This comprehensive report details a theoretical and implementation-ready framework for analyzing movement as a continuous system of attractors, utilizing stochastic differential equations, kinematic boundary conditions, and stability exponents to quantify the underlying structural architecture of competitive play.

2. Telemetry Architecture and the Sub-Frame Paradigm

The foundation of any sophisticated dynamical system analysis is the reliable acquisition of high-resolution, high-fidelity temporal data. In the context of the SSBM engine, the primary tool for modern data extraction is py-slippi, an open-source Python parser that directly interprets .slp files.¹ These replay files are generated by the Slippi Nintendont modification running on native hardware or via the Dolphin emulator.¹ The architecture of these files contains highly detailed "Pre-Frame" and "Post-Frame" state updates for every character, item, and projectile on every single frame of the game, which operates at a standard 59.94 fields per second.¹

2.1 Pre-Frame and Post-Frame Data Structures

The py-slippi library provides access to an incredibly rich set of mechanical variables that define the physical and logical state of the character. The parser separates the data into two distinct categories for each port on each frame: the pre-frame update and the post-frame update.⁵ The pre-frame data contains information collected right before controller inputs are processed to figure out the character's next action, capturing the raw joystick coordinates, C-stick coordinates, physical analog trigger depressions, and logical button flags.⁵

The post-frame data, conversely, represents the realized physical state of the character after the game engine has applied the physical inputs, resolved all environmental collisions, and calculated kinetic momentum.⁵ For the purposes of movement analysis and dynamical systems modeling, the post-frame data is the definitive source of truth, as it represents the character's actual physical manifestation within the coordinate space.

Data Field (py-slippi)	Unit / Data Type	Importance to Dynamical Systems Modeling

p.position.x, p.position.y	Engine Units (Floats)	Defines the exact continuous coordinate space of the agent on the 2D plane.
p.velocity.x, p.velocity.y	Units per Frame	Determines the instantaneous direction and magnitude of the system's state vector.
p.state	ActionState ID (Integer)	Categorizes the specific dynamical regime (e.g., Neutral vs. Punish).
p.state_age	Frames (Float)	Tracks the duration a specific movement pattern has been active, allowing for interpolation of animation states.
p.hit_stun	Frames (Float)	Indicates the presence of external deterministic forcing (the punish state).
p.direction	-1 (Left), 1 (Right)	Essential for calculating relative orientation, spacing, and asymmetrical hitbox reach.
p.airborne	Boolean	Defines whether the system is subject to ground friction or aerial drag constraints.

The ability of the library to report data with floating-point precision allows for the calculation of

exact derivative quantities such as acceleration and jerk, which are strictly necessary for identifying the invisible "forces" acting upon the player-character "particle". By aggregating these temporal observations across thousands of individual frames, analysts can reconstruct the global attractor that governs a player's spacing preferences and movement volatility.

3. The Kinematic Anchor: Root Bone Coordinates

To construct a valid mathematical manifold of player interaction, the system must first rigorously define the absolute position of the agents on the stage. However, a critical misunderstanding in amateur telemetry analysis involves the nature of the coordinate data. The `p.position` coordinates provided by the `py-slippi` post-frame object do not represent a "Center of Mass" in the traditional Newtonian physics sense, nor do they represent a simple geometric center of the character's 2D visual sprite.⁶

3.1 The Internal Coordinate Origin (Bone 0)

Instead, the spatial coordinates refer exclusively to the Root Bone (often designated in 3D modeling as Bone 0, or the Top Bone) of the character's skeletal armature.⁷ This Internal Coordinate Origin serves as the singular, definitive reference point that the game engine uses to track where a character is "located" within the global coordinate space of the stage.⁷ It acts as a master spatial position for the entire hierarchical skeleton.⁷

All other kinematic elements—including the torso, limbs, head, weapons, environmental collision detectors, hitboxes, and damageable hurtboxes—are strictly defined as spatial offsets branching outward from this central root point.⁶ In traditional 3D animation, the root bone is often stationed at the exact center of the world origin (0,0,0) before an animation begins, and moves as a single point to dictate the macroscopic locomotion of the entire character model.⁶

3.2 The Stability of the Root Anchor

In the context of dynamical telemetry extraction, tracking the Root Bone provides the most stable, unperturbed measure of a player's macro-trajectory.¹¹ If the system tracked the visual center of the sprite or a specific limb, the data would be massively polluted by the micro-oscillations caused by individual attack animations, landing crouches, or even idle breathing loops.¹¹ The Root Bone operates perfectly as the theoretical anchor for the dynamical system because it is completely intangible; it cannot be struck by an attack, it cannot hit an opponent, and it serves solely as the center of gravity for the player's kinetic inputs. While the Root Point anchors the character's agency, the physical realities of combat are delegated to secondary spatial offset structures.

4. Programmatic Extraction of the Neutral Manifold

Because the neutral game is fundamentally defined by the absence of an active punish state or a heavy positional disadvantage, specific internal Action State IDs must be utilized to filter the telemetry data before applying differential equations. The neutral game is composed of a very

specific subset of movement-focused states utilized to manipulate position relative to the opponent, bait attacks, and maintain kinetic readiness.

4.1 Action State Delineation

Using the enumerations provided by py-slippi, these neutral states are commonly defined within the integer ranges of 0 to 21, and 24 to 33.⁵ This spectrum of action states encompasses the entirety of standard locomotion and idle readiness.

Action State ID	Common Name	Role in the Neutral Dynamical System
0	Wait	The global attractor for an idle system; represents static equilibrium.
7	Walk	Low-velocity precision adjustment; linear drift without heavy momentum commitment.
15	Dash	High-velocity initial state; provides the initial kinetic impulse for movement chaos.
16	Run	Constant-velocity regime; sustained momentum across the stage manifold.
17	RunAdp	The transitional state required for pivot adjustments and run-canceling.

18	Squat	Crouch; heavily reduces the vertical hurtbox profile and stabilizes the potential well.
----	-------	---

4.2 Python Implementation for Telemetry Extraction

To isolate the neutral game, both players must be simultaneously occupying one of these predefined states. If one player is in hitstun (a punish state) or executing an attack (an offensive commitment), the true neutral equilibrium is broken. The following Python implementation demonstrates the precise extraction of the relative Euclidean distance between the Root Bones of two players strictly during these defined neutral frames:

Python

```
import os
import math
from slippi import Game

# Define the acceptable internal Action State IDs for neutral gameplay
# Ranges 0-21 and 24-33 capture all standard locomotion, jumps, and idle states
NEUTRAL_STATES = set(list(range(0, 22)) + list(range(24, 34)))

def is_neutral(post_frame_1, post_frame_2):
    """
    Evaluates whether a specific frame qualifies as a neutral interaction.
    Both agents must simultaneously occupy a neutral state to qualify the frame.
    """
    return (post_frame_1.state in NEUTRAL_STATES and
            post_frame_2.state in NEUTRAL_STATES)

def calculate_euclidean_distance(p1_x, p1_y, p2_x, p2_y):
    """
    Calculates the 2D spatial magnitude between the two Root Bones.
    This serves as the primary state variable D(t) on the relative manifold.
    """
    return math.sqrt((p1_x - p2_x)**2 + (p1_y - p2_y)**2)

def extract_neutral_distances(replay_dir):
    """
```

Parses a directory of .slp files and extracts a continuous array of neutral spacing distances for stochastic modeling.

```
"""
neutral_distances =

# Iterate through all replay files in the target directory
for filename in os.listdir(replay_dir):
    if not filename.endswith('.slp'):
        continue

    try:
        # Initialize the parser for the specific game file
        game = Game(os.path.join(replay_dir, filename))

        # Iterate over the parsed frames sequentially
        for frame in game.frames:
            # Filter valid ports containing active player data
            ports = [p for p in frame.ports if p is not None]
            if len(ports) < 2:
                continue

            # Extract post-frame telemetry for the primary characters
            p1, p2 = ports.leader.post, ports.leader.post

            # If the frame satisfies the neutral boundary conditions
            if is_neutral(p1, p2):
                dist = calculate_euclidean_distance(
                    p1.position.x, p1.position.y,
                    p2.position.x, p2.position.y
                )
                neutral_distances.append(dist)

    except Exception as e:
        # Silently pass malformed, incomplete, or corrupted replay files
        pass

return neutral_distances
```

By aggregating the output of the `extract_neutral_distances` function across massive datasets of professional matches, analysts are provided with a high-density, continuous scalar time-series. This array represents the realized, empirical spacing dynamics of a specific matchup, forming the foundational basis for further stochastic potential mapping and Lyapunov stability analysis.

5. Environmental Boundary Constraints and ECB Dynamics

While the Root Bone acts as the theoretical mathematical anchor for the dynamical system, it does not directly govern how the character physically interacts with the environment. The mathematical boundaries of the state space—such as the stage floors, lateral walls, and ceilings—are mediated through a separate, highly specialized collision geometry known as the Environment Collision Box (ECB).¹⁶

5.1 The Architecture of the ECB Diamond

The character's physical interaction with the stage topography is governed entirely by the ECB, which manifests as a diamond-shaped structure defined by exactly four distinct points (top, bottom, left, and right) roughly centered on the Root Bone.¹⁶ In debugging modes, the ECB from the previous frame is typically visualized as a dark orange diamond, while the projected post-collision ECB for the current frame is rendered in light orange.¹⁷

Each of the four points possesses a highly specific directional flag corresponding to environmental architecture.¹⁶ The left and right points exclusively calculate lateral wall collisions and solely restrict horizontal movement.¹⁶ The top point calculates ceiling collisions, arresting upward vertical momentum.¹⁶ Most critically, the bottom point of this diamond is the specific, singular coordinate that triggers floor detection, initiates landing mechanics, and forces state transitions from the aerial regime back to the grounded regime.¹⁶

When the engine calculates the post-frame physics, the ECB's position is projected linearly along the character's velocity vector. If the projected path of the bottom point intersects with a stage floor's line segment, the system resolves the collision, immediately halts downward vertical velocity, and alters the Action State of the character (e.g., transitioning from an aerial state into landing lag).¹⁶ If the character's movement creates massive spatial displacements in a single frame, the engine splits the translation into multiple discrete steps and performs continuous linecast collision checks for each step to prevent characters from clipping through thin surfaces.¹⁶

5.2 ECB Interpolation and Temporal Freezing Mechanics

The dynamic nature of the ECB introduces severe non-linearities into the system's boundary conditions. Every single animation in the game possesses a unique ECB pattern, shifting the four boundary points to different spatial offsets relative to the Root Bone.¹⁶ To prevent catastrophic clipping or unintended teleportation through stage geometry during rapid animation changes, the engine utilizes a mechanic known as ECB freezing.¹⁶

When a character transitions from a grounded state to an aerial state, or initiates a double jump, the ECB point responsible for floor detection is "frozen" relative to the Root Bone's position for exactly 10 frames.¹⁶ If this temporal delay were not implemented, contorting aerial

animations (such as Falco's back aerial) would instantly elevate the bottom ECB point the moment the animation began, causing the character to inadvertently phase upwards through semi-permeable platforms immediately after dropping through them.¹⁶

Crucially, this 10-frame timer is tied to the engine's internal clock, not the character's animation clock. This means that hitlag (the freeze frames that occur when an attack connects) continues to count down the 10-frame ECB freeze.¹⁶ This hitlag-dependent freezing mechanism acts as a delayed-differential constraint in the dynamical system, meaning the system's current environmental bounding box is a complex function of its state history from several frames prior.¹⁶

6. Advanced ECB Exploitation: NILs and Aerial Interrupts

Because the ECB is highly mutable based on animation data, high-level players intentionally manipulate these boundary constraints to generate instantaneous, mathematically abrupt positional changes. By rapidly altering their ECB diamond to prematurely intersect with a platform, players can bypass standard velocity curves.

6.1 No-Impact Landings (NIL)

A No-Impact Land (NIL) occurs when a character lands on a platform with minimal downward velocity, allowing them to instantly enter the Wait state without suffering standard landing lag.¹⁷ Players can force a NIL by utilizing specific aerials or jump arcs that possess ECB patterns with severe downward extensions.¹⁷ If a character jumps slightly above a platform and their jumping animation features an ECB that abruptly extends downward below the previous frame's geometry, the bottom ECB point will intersect the platform while the character still maintains upward or neutral vertical velocity, instantly triggering a NIL.¹⁷

6.2 Aerial Interrupts and Platform Warping

Similarly, Aerial Interrupts exploit the immediate downward extension of specific aerial attack animations.¹⁷ For example, certain characters can perform a full hop that places their Root Bone just above a platform. By executing an aerial attack at the absolute apex, the character's ECB diamond snaps downward to match the new animation's parameters. If this new bottom point crosses the platform threshold, the game instantly registers a landing event, canceling the aerial attack on its first frame and forcing the character onto the platform.¹⁷

Platform Warping takes this manipulation further by exploiting the 10-frame freeze mechanic. If a player drops through a platform, the bottom ECB point is frozen.¹⁷ If they immediately execute an aerial attack that strikes a shielded opponent, the resulting hitlag will cause the 10-frame freeze timer to expire while the characters are frozen in space.¹⁷ Once the timer expires, the ECB updates to the aerial animation's natural position, which may be significantly higher. If this updated ECB position places the bottom point back above the platform, the game instantly

resolves the collision, warping the character upward onto the surface the moment hitlag ends.¹⁷ These extreme non-linearities must be accounted for in dynamical modeling, as they allow characters to violate standard ballistic trajectories.

7. The Geometry of Agency: Hitboxes and Hurtbox Capsules

While the ECB completely dictates interaction with the static environmental stage, player-to-player interaction is governed by an entirely separate, vastly more complex matrix of collision geometries. The Root Point operates as the anchor of the character's physical agency, but it is entirely intangible to opponent attacks.¹⁹

7.1 Bone Offsets and Defensive Hurtboxes

Damage detection is registered through hurtboxes (officially termed "reception judgements"). These are strictly defined as spherical or capsule-shaped volumetric "bubbles" (cylinders capped with spheres) attached to specific anatomical nodes within the character's skeletal hierarchy, such as the hand, the torso, or the foot.¹⁹ Because these hurtboxes are children of specific bones, their position in global world space is calculated continuously as the sum of the Root Bone's coordinates plus the animation-driven bone offset.¹⁹ These yellow collision bubbles represent the actual damageable surface area of the character.

7.2 Hurtbox Shifting and Spatial Deception

During complex locomotive animations or attacks, characters frequently contort, lean, retract their limbs, or stretch. This biomechanical phenomenon is categorized mathematically and competitively as "hurtbox shifting".¹⁹ Hurtbox shifting allows a character to drastically alter the effective range of a physical interaction without generating any actual displacement of their primary Root Coordinate.¹⁹

For example, a character utilizing a low-profile animation (such as a squat, or the backward contortion of a specific dash-dance turn) may dramatically shift their upper torso and head hurtboxes backward along the horizontal axis.¹⁹ This mechanical nuance creates significant, highly deceptive distortions in the relative distance mapping. Two players may maintain mathematically identical Root Bone distances on the relative manifold, but their effective vulnerability may fluctuate wildly depending on the exact sub-frame phase of their respective animation cycles.¹⁹ A character leaning backward may be completely safe from an incoming attack, while a character leaning forward at the exact same Root Distance will suffer a catastrophic hit.¹⁹

7.3 The Mathematical Construction of Threat Bubbles

Conversely, offensive attacks generate hitboxes (red collision bubbles) that exist for specific, active frames of an animation.¹⁹ The physical offsets of these hitboxes give rise to the concept

of "threat zones" or "burst ranges"—the actual spatial regions where a player can physically influence the state of the game and deal damage.²²

A player's threat bubble is not a static shape; it is a dynamic, multidimensional geometric manifold representing the maximum spatial reach of all their available offensive hitboxes over a given time horizon.²² This calculation inherently accounts for their current instantaneous velocity, their traction, and the frame-startup data of their available action states.²² Therefore, in high-level competitive analysis, the neutral game is not merely an optimization of Root Bone distance, but the continuous intersection and evasion of these invisible, rapidly fluctuating threat bubbles.

8. Collision Mechanics and Topological Bifurcations

The neutral game shatters when a player successfully circumvents the opponent's spatial defenses, forcing a defensive commitment or landing a direct hit. Topologically, this event is classified as a dynamical bifurcation—a sudden, violent qualitative change in the fundamental behavior of the system, transitioning from chaotic spacing to a deterministic punish state. This bifurcation physically occurs at the exact computational moment that an offensive hitbox bubble overlaps a vulnerable defensive hurtbox bubble.

8.1 The Law of High and Low Priority: Clanking

When threat bubbles intersect, the engine must immediately resolve the interaction before rendering the next frame. If a hitbox intersects a hurtbox, standard damage and knockback formulas are applied. However, if two opposing hitboxes intersect simultaneously in space, the resulting bifurcation depends entirely on the engine's strict priority rules.²³

Grounded attacks are strictly governed by the "Law of High and Low Priority".²⁴ If two grounded hitboxes collide, the engine evaluates the base damage differential between the two attacks. If the difference in base damage is 9% or less, the attacks enter a state known as a "clank".²³ Mathematically, a clank acts as a massive, instantaneous dampening force on the dynamical system. Both attacks are immediately terminated, their respective momentum vectors are heavily scrubbed, and both characters are forced into a rebound animation characterized by a distinct visual flash and audio cue.²³

The duration of this rebound animation is highly deterministic and scales directly with the damage the moves would have dealt. The specific calculation for rebound frames (R) based on damage (d) is defined by the function:

$$R = \lceil 0.559 \times (d + 10) \rceil$$

This rebound state dissipates the kinetic energy of the interaction, safely returning the system back toward the neutral attractor.²³

8.2 Hitbox Override and Priority Annihilation

If the collision occurs and the damage difference exceeds the 9% threshold, the stronger hitbox completely out-prioritizes the weaker one.²³ The weaker hitbox is instantly annihilated, and its user is cast into the rebound animation.²³ Crucially, the stronger attack is completely uninterrupted; it continues along its trajectory and will strike the opponent's hurtbox if it remains within range, forcing a massive bifurcation directly into the punish state.²³ For example, Bowser's heavy forward smash (23%) will completely out-prioritize and crush a weak tilt attack, but will mathematically clank against Samus's fully charged Charge Shot (25%) due to falling within the 9% differential window.²³

Furthermore, if an attack contains multiple hitboxes with varying properties that connect on the exact same frame, the engine utilizes Hitbox IDs to resolve the collision. The hitbox with the lowest ID value (e.g., ID 0) registers first, granting it supreme local priority.¹⁹ This structural hierarchy is absolutely imperative for defining "sweetspots" (high-damage, high-knockback nodes) versus "sourspots" (low-damage peripheral nodes) on complex attacks like Marth's sword strikes.¹⁹

8.3 Aerial Interactions and Transcendent Priority

Unlike grounded interactions, aerial hitboxes operate under an entirely different subset of topological rules. Normal aerial attacks do not clank under any circumstances, regardless of their damage output or priority differential.²⁴ If two aerial hitboxes overlap in physical space, they simply pass directly through one another.²⁵ Consequently, if the corresponding hurtboxes are within range, both characters will simultaneously absorb the kinetic energy of the opponent's attack, resulting in a mathematical "trade".²⁴

Furthermore, specific moves are permanently programmed with "Transcendent Priority".²⁴ Hitboxes bearing this property—such as energy projectiles, Fox's laser blasts, or Bowser's Neutral-B fire breath—are completely removed from the priority calculation matrix.²⁴ They cannot clank with other moves, nor can they be cancelled by them.²⁵ Transcendent priority represents an absolute forcing function in the dynamical system; it cannot be neutralized by an opposing force, guaranteeing that it will either miss entirely or trigger a state-change bifurcation upon contacting a hurtbox.²⁵

9. Mathematical Formulation of the Relative State Space

To transition from raw engine mechanics to high-level dynamical systems theory, the interactions extracted via telemetry must be formalized into a rigorous mathematical space. Traditional game analysis relies heavily on absolute stage coordinates; however, because the neutral game is functionally defined by the spatial relationship between the two agents, absolute position is mathematically secondary.

9.1 The Relative Manifold Projection

Let the state of the game at any discrete frame t be defined by a high-dimensional state vector $X(t) \in \mathbb{R}^n$. For a standard one-versus-one interaction, the continuous mechanical variables for each player $i \in \{1, 2\}$ consist of their two-dimensional Root Bone position $s_i(t)$ and their instantaneous velocity vector $v_i(t)$:

$$s_i(t) = \begin{bmatrix} x_i(t) \\ y_i(t) \\ v_{ix}(t) \\ v_{iy}(t) \end{bmatrix}^T$$

To simplify this complex 8-dimensional space and capture the core kinetic essence of the neutral interaction, $X(t)$ is projected onto a reduced-order Relative Manifold M . The primary state variable on this manifold is the scalar Euclidean distance $D(t)$ between the Root Bones of the two agents:

$$D(t) = \sqrt{(x_1(t) - x_2(t))^2 + (y_1(t) - y_2(t))^2}$$

The temporal derivative of this distance provides the relative closing velocity $V_{rel}(t)$, which acts as the critical secondary state variable for modeling kinetic pressure:

$$V_{rel}(t) = \frac{dD(t)}{dt} = \frac{(x_1 - x_2)(v_{1x} - v_{2x}) + (y_1 - y_2)(v_{1y} - v_{2y})}{D(t)}$$

By systematically studying the phase portrait of (D, V_{rel}) , the continuous, high-speed adjustments of the neutral game are translated into an observable orbit around a central region of the state space. When players engage in active, respectful spacing, $D(t)$ oscillates within a highly specific range, while $V_{rel}(t)$ fluctuates symmetrically around zero, indicating that neither player is fully committing to a massive closing vector.

10. Attractor Dynamics and Langevin Spacing Potentials

During the neutral phase, players continuously attempt to maintain a specific equilibrium distance D_{opt} . At this optimal distance, they can maximize their neurological reaction time to

incoming attacks while keeping their own offensive threat bubbles overlapping the opponent's defensive hurtboxes. In the terminology of non-linear dynamics, this behavioral targeting forms a Strange Attractor—a localized set of states toward which the dynamic system naturally evolves despite continuous external perturbation.¹

10.1 The Langevin Model for Spacing

Player movement on the relative manifold can be modeled mathematically using a stochastic differential equation known as the Langevin equation, a formulation originally developed to model the random walk of particles in fluid mechanics.¹ This model elegantly treats the players' conscious, strategic intent to maintain optimal spacing as a deterministic restoring force, while their continuous kinetic micro-adjustments, execution imperfections, and human reaction delays are treated as stochastic noise.¹

The evolution of the relative distance D_t over time is mathematically described as:

$$dD_t = -\nabla U(D_t)dt + \sigma dW_t$$

Where:

- $U(D)$ represents the "potential well" or the spatial basin of attraction, representing the psychological and strategic "gravity" pulling the players toward safety.
- σ is the diffusion coefficient, representing the physical magnitude of movement volatility, kinetic energy, or "twitchiness".¹
- W_t represents a standard Wiener process, modeling the Gaussian noise inherent in human biomechanical inputs, controller polling variances, and system processing delays.¹

10.2 Emergence of the Potential Well from Threat

Crucially, the potential well $U(D)$ is not hardcoded into the game engine; it is an emergent property of the matchup, the stage geometry, and the unique psychological profiles of the competitors. The theoretical framework developed herein treats the Root Point as the anchor of agency, but the formation of $U(D)$ fully encapsulates the immense complexities of the aforementioned hitboxes, hurtbox shifting, and threat bubbles.

A player does not instinctively base their spacing calculations on the opponent's invisible Root Bone coordinate; they space based on the outermost visible arc of the opponent's physical sword, or the maximum forward extension of an aerial kick. Therefore, the local minima in the potential energy landscape $U(D)$ precisely reflect the exact distances where these specific hitbox/hurtbox offsets are perfectly optimized for either defensive safety or offensive initiation.

Using the empirical telemetry extracted via the `extract_neutral_distances` Python function, the true shape of $U(D)$ can be mathematically reconstructed from the probability density function $\rho(D)$ of the distances maintained during thousands of neutral frames. Applying the Boltzmann distribution derived from statistical mechanics, the potential well is calculated as inversely proportional to the log-likelihood of the observed distances:

$$U(D) = -\ln(\rho(D))$$

Deep, narrow minima in the resulting $U(D)$ plot indicate a highly rigid, strict spacing preference typical of highly defensive, precise playstyles. Broad, shallow potential wells indicate a highly volatile, high-variance approach to neutral interactions, where the player is comfortable traversing wide swathes of the relative manifold.

11. Bifurcations and Lyapunov Exponents

While potential wells accurately map where the system *desires* to be located, stability exponents are required to mathematically quantify how resilient those desired states are to kinetic perturbation. To mathematically distinguish between the chaotic "neutral" state and the deterministic "punish" state, the framework measures the maximum Lyapunov exponent (λ), a concept derived from chaos theory which calculates the average rate of exponential divergence or convergence of nearby trajectories in the multidimensional phase space.¹

For an infinitesimally small spatial perturbation δX , the maximum Lyapunov exponent is strictly defined as:

$$\lambda = \lim_{t \rightarrow \infty} \lim_{\|\delta X(0)\| \rightarrow 0} \frac{1}{t} \ln \frac{\|\delta X(t)\|}{\|\delta X(0)\|}$$

11.1 Bounded Chaos During Neutral ($\lambda > 0$)

During the neutral game, the interaction between the two highly skilled agents exhibits bounded chaos.¹ Even when two players are hovering completely still near their optimal equilibrium distance D_{opt} , their relentless kinetic micro-movements—such as high-speed dash-dancing, wavelanding, and hurtbox shifting via rapid crouching—cause their trajectory paths to diverge rapidly if even slightly perturbed.

This extreme sensitivity to initial conditions yields a distinctly positive maximum Lyapunov exponent ($\lambda > 0$). A positive λ serves as the rigorous mathematical basis for the competitive concept of "mix-ups". Because the state trajectories diverge at an exponential rate, it is

theoretically and practically impossible for an opponent to predict a player's exact coordinate position or hurtbox orientation several frames into the future based merely on their current velocity vector. The physical movement is undeniably chaotic, but it remains bounded by the global "strange attractor" of mutual threat ranges, preventing the players from simply moving infinitely far apart and forcing engagement.

11.2 The Collapse into Determinism ($\lambda \leq 0$)

The defining characteristic of a successful Opening (a landed hit) is the sudden, catastrophic collapse of this chaotic agency. When an offensive hitbox successfully overlaps a defensive hurtbox without clanking, the victim enters the "hitstun" state.¹ During hitstun, the victim's controller inputs are heavily disabled, and their physical coordinate trajectory is dictated entirely by the engine's internal knockback formulas, weight coefficients, and gravity modifiers.¹

Topologically, during this punish state, the maximum Lyapunov exponent drops precipitously to zero or becomes negative ($\lambda \leq 0$). The system's trajectories are no longer diverging based on bi-directional human agency; they are converging along a strict, highly deterministic ballistic arc. The exact parameters of this deterministic state are dictated entirely by the specific hitbox that connected. Different hitboxes on the exact same character model possess drastically different base knockback values, knockback growth scaling, and highly specific launch angles.¹ These invisible parameters perfectly determine the mathematical trajectory of the subsequent combo tree. The sudden collapse of λ provides a pure, objective metric for identifying who is "winning neutral"—it is the precise moment when the chaotic, bi-directional agency of the two-player interaction is violently overridden by the singular deterministic control of the attacking player.

12. Character-Specific Physics as Control Parameters

The specific geometric topology of the attractors, the resulting depth of the potential wells, and the average magnitude of the Lyapunov exponents are all intrinsically tied to the internal physical attributes hardcoded into each of the fighters.¹ The SSBM engine queries a unique table of physics constants for each character on every single frame, which mathematically act as the fundamental control parameters of our dynamical system.

12.1 Traction and Friction Dynamics

In the specific context of the Langevin model, a character's internal "Traction" statistic acts directly as the macroscopic damping coefficient (γ) or friction force.¹ Traction rigorously determines the rate of velocity decay when analog stick inputs are released, defining how quickly a character can arrest their kinetic momentum.¹

Character Archetype	Engine Traction Value	Dynamical System Interpretation
Luigi	0.025	Highly underdamped; experiences extreme slide distances. Results in high-volatility, wide, flattened potential wells.
Ice Climbers	0.035	Underdamped; low stability in neutral, reliant on desynchronized hitboxes to artificially extend threat bubbles.
Mewtwo	0.040	Transitional underdamped; slippery but allows for moderate micro-spacing control.
Marth / Roy	0.060	Balanced damping; produces standard, highly stable and predictable neutral attractors.
Fox / Falco	0.080	Overdamped; rapid deceleration allows for hyper-precision spacing, tight dash-dances, and deeply carved potential wells.
Peach	0.100	Highly overdamped; character practically "sticks" to the floor

		instantly, resulting in near-static equilibrium states.
--	--	---

Characters saddled with exceptionally low traction values (e.g., Luigi) are highly prone to sliding, yielding a flattened, excessively wide potential well $U(D)$, as the player struggles to arrest momentum instantaneously and must constantly preempt their braking. Conversely, extremely high-traction characters like Peach can halt their velocity vectors almost instantly, creating deep, narrow potential wells that represent static stability, allowing them to precisely park their hurtboxes just millimeters outside an opponent's threat bubble.

12.2 Vertical Dynamics: Gravity and Terminal Fall Speed

While traction completely dominates the horizontal plane of the relative manifold, vertical kinematics are governed by the specific variables of Gravity and Terminal Fall Speed.¹ Gravity acts as a constant downward drift forcing the state vector back toward the floor boundary (the ECB bottom point).¹

For "floaty" characters such as Jigglypuff, the vertical potential well is exceedingly shallow, permitting the character to traverse the upper aerial limits of the state space for extended periods, bypassing grounded interactions almost entirely. For "fast-fallers" like Fox or Falco, the potential well is aggressively steep, forcibly pulling the character back to the stage surface and resulting in a high-frequency vertical oscillation as they rapidly cycle between aerial strikes and grounded movement. Furthermore, players can manipulate the system's gravity parameter mid-trajectory via "fast-falling"—a specialized input that acts as a mathematical step-function, abruptly increasing the downward velocity vector to maximum terminal velocity and severely distorting the predictability of the aerial trajectory.¹

13. Metastability, Fractal Dimensions, and Edge Cases

Beyond simple, singular potential wells, high-level player movement often results in highly complex, multi-modal potentials. A purely resting state is a massive vulnerability in SSBM; thus, the neutral game relies on the execution of sustained metastability.¹

13.1 The Multi-Modal Potential Landscape

Metastability dictates that a dynamical system remains stable within a localized basin for a protracted period, but exists in a state of high tension where it can be instantly tipped into an adjacent state via a microscopic perturbation. In a competitive match, an oscillating dash-dance is a physical manifestation of this metastability; the player is constantly vibrating their Root Bone coordinate, injecting kinetic energy into the system to prevent it from settling into a static, predictable equilibrium.¹

For an elite-level player, the spatial potential well $U(D)$ exhibits two distinct, multi-modal basins:

1. **The Defensive Basin:** A deep, highly stable well located far outside the opponent's threat bubble, allowing ample time to process visual stimuli and react to encroaching hitboxes without fear of instantaneous bifurcations.
2. **The Offensive Basin:** A secondary, somewhat shallower well hovering aggressively on the absolute outer edge of the opponent's hurtbox shifting range. It carries a drastically higher risk of bifurcation (being hit) but mathematically maximizes the player's own follow-up potential and pressure.¹

The defining hallmark of competitive mastery is the ability to orchestrate a "random walk" between these two basins, smoothly altering the Root Coordinate distance without lingering long enough in the transitional space to be intercepted by a transcendent hitbox.

13.2 Fractal Dimensions of Movement

When mapped fully over the course of a professional match, the phase portrait of a player often resembles legendary chaotic geometries, such as the Lorenz attractor or the Hénon map.¹ Because these specific orbits never perfectly repeat—due to the continuous injection of human execution noise, minor analog polling variations, and strategic adaptations—they possess non-integer, fractal dimensions. By computationally calculating the Correlation

Dimension (D_2) of the distance time-series, analysts can rigorously quantify the objective complexity of a player's movement.

A remarkably low D_2 value (≈ 1) indicates rigid, repetitive, and ultimately solvable movement patterns that can be easily predicted and exploited. A high D_2 value (> 2) mathematically proves that the player is successfully utilizing deeply layered mix-ups, recursively nesting varied dash lengths, precisely timed wavelands, and ECB manipulations to ensure their trajectory fills the state space in an entirely unpredictable manner.¹

13.3 Edge Cases: Port Priority and RNG Integration

While the dynamical system is primarily deterministic concerning physics, specific edge cases inject extreme variance that must be accounted for in stochastic models.

In rare boundary cases where two identical interactions (such as simultaneous grab attempts on the exact same frame) require resolution, the system experiences a temporal paradox.²⁵ The engine resolves this by referencing "Port Priority"—a hardcoded hierarchy granting absolute precedence to the player physically plugged into the lower-numbered controller port (closer to Port 1).²⁵ In such edge cases, the bifurcation of the system is entirely dependent on hardware configuration rather than spatial dynamics.²⁵ Furthermore, port priority mathematically alters the hitstun on all throws, artificially subtracting one frame of hitstun from any of the lower port

player's throws by registering the release frame as hitstun.²⁵

Additionally, true Random Number Generation (RNG) is embedded in specific character kits, introducing non-Gaussian noise into the system. For instance, the character Peach utilizes a `get_random_int` function to determine the exact properties of her turnip projectiles.³⁵ The mathematical probability dictates a 1-in-58 chance of pulling a catastrophic "stitch-face" turnip, which possesses vastly superior damage and knockback.³⁵ The sudden introduction of this item into the phase space massively expands Peach's threat bubble and instantly deepens her offensive potential well, forcing the opponent's system to immediately readapt to a newly generated, highly volatile attractor state.

14. Conclusion

The rigorous framework outlined in this analytical report decisively demonstrates that *Super Smash Bros. Melee* functions less like a traditional discrete video game and more like a continuous, highly complex biomechanical and physical system. By anchoring the analysis to the programmatic reality of the Root Bone coordinates, explicitly defining the environmental boundary constraints via the Environment Collision Box, and accurately mapping the spatial reach of dynamic hitboxes and contorting hurtboxes, a mathematically rigorous representation of player interaction emerges.

Through the sophisticated lens of stochastic differential equations and chaos theory, the elusive "neutral game" is permanently demystified. It is fundamentally a state of bounded kinetic chaos, where highly skilled human agents navigate a multidimensional landscape of potential energy wells, seeking to maintain dynamic metastability while simultaneously attempting to force their opponent into a catastrophic topological bifurcation. The precise computational measurement of Lyapunov exponents provides the long-sought objective boundary between the agency of spacing and the determinism of the combo tree.

As telemetry extraction mechanisms like `py-slippi` continue to advance in resolution and efficiency, the application of non-linear dynamics, kernel density estimation, and fractal geometry will rapidly become the definitive standard for understanding, predicting, and ultimately optimizing the profound complexities of high-velocity human competition within simulated environments.

Works cited

1. Dynamical Systems Analysis of Slippi Data (1).pdf
2. INSANE Glitches in Super Smash Bros. Melee - YouTube, accessed March 11, 2026, <https://www.youtube.com/watch?v=9KZPoDHZzFY>
3. SSBM Terminology : r/smashbros - Reddit, accessed March 11, 2026, https://www.reddit.com/r/smashbros/comments/2y2ufo/ssbm_terminology/
4. hohav/py-slippi: Python library for parsing SSBM replay files - GitHub, accessed March 11, 2026, <https://github.com/hohav/py-slippi>
5. slippi package — py-slippi 1.5.1 documentation - Read the Docs, accessed March

- 11, 2026, <https://py-slippi.readthedocs.io/en/latest/source/slippi.html>
6. Proper Root Bone Placement - CG Cookie, accessed March 11, 2026, <https://cgcookie.com/community/13377-proper-root-bone-placement>
 7. Root- What exactly does it do - Spine Forum, accessed March 11, 2026, <https://esotericsoftware.com/forum/d/316-root-what-exactly-does-it-do>
 8. Physiologically based mathematical model of transduction ... - PubMed, accessed March 11, 2026, <https://pubmed.ncbi.nlm.nih.gov/21336969/>
 9. Should a root bone position be ground level? (Both in ... - Reddit, accessed March 11, 2026, https://www.reddit.com/r/unrealengine/comments/u2wqjn/should_a_root_bone_position_be_ground_level_both/
 10. Applying Root Motion - Spine Forum, accessed March 11, 2026, <https://esotericsoftware.com/forum/d/1364-applying-root-motion/18>
 11. Shifting the Root Bone Position after Animation, accessed March 11, 2026, <https://blender.stackexchange.com/questions/284521/shifting-the-root-bone-position-after-animation>
 12. Object IPO or root bone for position? - Blender Artists Community, accessed March 11, 2026, <https://blenderartists.org/t/object-ipo-or-root-bone-for-position/317273>
 13. py-slippi · PyPI, accessed March 11, 2026, <https://pypi.org/project/py-slippi/1.2.0/>
 14. Python Module Index - py-slippi - Read the Docs, accessed March 11, 2026, <https://py-slippi.readthedocs.io/en/latest/py-modindex.html>
 15. Super Smash Bros. Melee/Stages - StrategyWiki, accessed March 11, 2026, https://strategywiki.org/wiki/Super_Smash_Bros._Melee/Stages
 16. Is there any resource that outlines Melee's ECB logic? : r/SSBM, accessed March 11, 2026, https://www.reddit.com/r/SSBM/comments/q5nb2g/is_there_any_resource_that_outlines_melees_ecb/
 17. SSBM | The mechanics behind advanced platform tech - YouTube, accessed March 11, 2026, <https://www.youtube.com/watch?v=TtiDd5cQWJc>
 18. Debug menu (SSBM)/DEVELOP mode - SmashWiki, accessed March 11, 2026, [https://www.ssbwiki.com/Debug_menu_\(SSBM\)/DEVELOP_mode](https://www.ssbwiki.com/Debug_menu_(SSBM)/DEVELOP_mode)
 19. Hitbox - SmashWiki, the Super Smash Bros. wiki, accessed March 11, 2026, <https://www.ssbwiki.com/Hitbox>
 20. Explaining hurtbox shifting! (All smash games) - YouTube, accessed March 11, 2026, <https://www.youtube.com/watch?v=jlvc27cdWjg>
 21. Pyra & Mythra Hurtbox Shifting EXPLAINED - Smash Ultimate, accessed March 11, 2026, https://www.youtube.com/watch?v=OzRQK_KZbY4
 22. An analysis of hitboxes in motion | Melee Smash - YouTube, accessed March 11, 2026, <https://www.youtube.com/watch?v=ezQZdjvwbTk>
 23. Priority - SmashWiki, the Super Smash Bros. wiki, accessed March 11, 2026, <https://www.ssbwiki.com/Priority>
 24. Priority | Smashpedia | Fandom, accessed March 11, 2026, <https://supersmashbros.fandom.com/wiki/Priority>
 25. All About Priority In Smash Melee - Dignitas, accessed March 11, 2026,

- <https://dignitas.gg/articles/blogs/smash/12745/all-about-priority-in-smash-melee>
26. Hitbox priority : r/smashbros - Reddit, accessed March 11, 2026, https://www.reddit.com/r/smashbros/comments/2xjgor/hitbox_priority/
 27. ATrie Group Join: A Parallel Star Group Join and Aggregation for In, accessed March 11, 2026, <https://www.computer.org/csdl/journal/bd/2022/04/09124683/1kVbyfM4qzu>
 28. Let's analyze some ultimate hitboxes (Smash Ultimate) - YouTube, accessed March 11, 2026, <https://www.youtube.com/watch?v=nkzWPQshYGQ>
 29. Explaining Game & Watch's Weird Hitboxes (Smash Ultimate), accessed March 11, 2026, <https://www.youtube.com/watch?v=ec6Gcy4WyhM>
 30. Why is skeletal mesh origin at root bone position and how do I fix it?, accessed March 11, 2026, <https://forums.unrealengine.com/t/why-is-skeletal-mesh-origin-at-root-bone-position-and-how-do-i-fix-it/662907>
 31. Attach mesh to bone => wrong position - Questions - Babylon.js, accessed March 11, 2026, <https://forum.babylonjs.com/t/attach-mesh-to-bone-wrong-position/6910>
 32. List of regional version differences (SSBM) - SmashWiki, accessed March 11, 2026, [https://www.ssbwiki.com/List_of_regional_version_differences_\(SSBM\)](https://www.ssbwiki.com/List_of_regional_version_differences_(SSBM))
 33. Melee 1.03 is NOW ON SLIPPI! : r/SSBM - Reddit, accessed March 11, 2026, https://www.reddit.com/r/SSBM/comments/uezpau/melee_103_is_now_on_slippi/
 34. How Port Priority Can Affect Your Game - Super Smash Bros. Melee, accessed March 11, 2026, <https://www.youtube.com/watch?v=kTNI03-1fOM>
 35. The Basics of RNG in Melee : r/SSBM - Reddit, accessed March 11, 2026, https://www.reddit.com/r/SSBM/comments/71gn1d/the_basics_of_rng_in_melee/